
Artificial Intelligence

BS (CS) Spring 2026

Lab 04 Tasks



Learning Objectives:

1. Uninformed Search
2. Breadth-First Search
3. Depth-First Search
4. Uniform Cost Search
5. Tree Search

Lab Tasks

Submission Instructions

1. Create one notebook file for all questions with name *ROLLNO_SEC_LAB3* e.g. **i23XXXX_A_LAB04**
2. Compress your **.ipynb** file along with output **screenshots** directly into a **.zip file**.
3. Now you have to submit this zipped file on Google Classroom under your section heading.
4. If you don't follow the above-mentioned submission instruction, you will be marked **zero**.

Plagiarism in the Lab Task will result in **zero** marks in the whole category.

Questions

Q1. A city's traffic control system wants to determine the minimum number of intersections a driver must pass through to get from one intersection to another. Each intersection is a node in a graph, and roads connecting intersections are edges.

Write a Python function that uses BFS to find the shortest route from the starting intersection to the destination intersection.

Input:

- `graph` – adjacency list representing intersections as nodes
- `start` – starting intersection
- `end` – destination intersection

Output:

- List of intersections forming the shortest path, or `None` if unreachable

Example

```
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}
```

```
start = 'A'
end = 'F'

# Shortest path:
['A', 'C', 'F']
```

Q2. A cybersecurity analyst wants to simulate how a virus spreads in a computer network. Each computer is a node, and direct network connections are edges.

Write a Python function that uses DFS to determine all computers that can be infected starting from a given computer.

Input:

- graph – adjacency list of computers
- start – initially infected computer

Output:

- List of computers that get infected (DFS order)

Example:

```
graph = {
    'C1': ['C2', 'C3'],
    'C2': ['C1', 'C4'],
    'C3': ['C1'],
    'C4': ['C2', 'C5'],
    'C5': ['C4']
}

start = 'C1'

# One possible DFS output:
['C1', 'C2', 'C4', 'C5', 'C3']
```

Q3. In a smart city, emergency vehicles such as ambulances must be routed efficiently from a control center to a hospital. However, road conditions are dynamic, and ongoing road construction can temporarily increase travel costs.

Problem Setup

The city is modeled as a weighted graph where:

- Nodes represent intersections.
- Edges represent roads connecting intersections.
- Each road has a base travel cost.
- Roads under active construction incur an additional random cost between 0–3 units at runtime.

Road Network Details

Intersection 1	Intersection 2	Base Travel Cost	Construction Status
S	A	4	Inactive
S	B	7	Active
A	C	3	Inactive
B	C	2	Active
B	D	6	Inactive
C	E	5	Active
D	E	4	Inactive
E	H	6	Inactive

Task

- Objective: Find the least-cost path from S to H.
- Roads marked Active add a random cost (0–3) during traversal.
- Use Uniform Cost Search (UCS) to determine the optimal path.

Expected Output

```
city_graph = {
    'S': [('A', 4, "no"), ('B', 7, "yes")],
    'A': [('C', 3, "no")],
    'B': [('C', 2, "yes"), ('D', 6, "no")],
    'C': [('E', 5, "yes")],
    'D': [('E', 4, "no")],
    'E': [('H', 6, "no")]
}

# Start and goal nodes
start_intersection = 'S'
destination_intersection = 'H'
```

Path : $S \rightarrow A \rightarrow C \rightarrow E \rightarrow H$ (cost: $4 + 3 + 5 + 6 = 18$)

Note: Cost may vary a little due to construction randomness